

MULTIMEDIA UNIVERSITY OF KENYA

Data Structures and Algorithms — Task One

MATHEW CHACHA CIT-227-088/2025

A Modern Parking Management System

1. Introduction

The client requires an automated parking system for operations in Kenya. Drivers must see a visual display of available parking slots before entry. The system records each vehicle on arrival, and on exit it automatically calculates the total time spent and the amount payable. Payment can be made by M-Pesa, card, or cash, and the barrier opens only once payment is confirmed. Management can adjust parking rates at any time, and every completed payment is recorded for reconciliation and VAT purposes. This document sets out the system modules, the algorithm for each module, the data structures used, and the design of a dynamic database for the scenario.

2. System Modules

Analysing the requirements, the system is broken down into seven modules:

Module	Purpose
1. Slot Display Module	Shows drivers available parking bays before entry
2. Vehicle Entry Module	Records a vehicle on arrival and assigns a bay
3. Fee Calculation Module	Computes time spent and fee owed on exit
4. Payment Module	Captures the payment method and confirms payment
5. Barrier Control Module	Opens the barrier once payment is confirmed
6. Rate Management Module	Lets management update parking rates at any time
7. Audit and Reporting Module	Keeps a permanent transaction log for reconciliation and VAT

3. Algorithms per Module

3.1 Slot Display Module

Shows drivers how many bays are free before they attempt to enter.

1. Load slot data from the JSON data store.
2. For each slot, check its status (Free or Occupied).
3. Count the total number of free slots.
4. Send the slot list and free count to the display page.
5. Display the result to the driver before entry.

3.2 Vehicle Entry Module

Records a vehicle when it arrives and assigns it an available bay.

1. Driver requests entry and provides a number plate.
2. If the plate is empty, display an error and stop.
3. If the plate is already parked inside, display an error and stop.
4. Check the slot list for a slot marked Free.
5. If no free slot exists, display a “Parking Full” message and stop.
6. Otherwise, assign the first available slot to the vehicle.
7. Record the plate number, assigned slot, and entry time.
8. Mark the assigned slot as Occupied and save the updated data.
9. Open the entry barrier.

3.3 Fee Calculation Module

Works out how long a vehicle has been parked and what it owes, using rates that management can change at any time.

1. Driver requests exit and provides the number plate.
2. If no matching record is found, display an error and stop.
3. Retrieve the entry time and take the current time as the exit time.
4. Calculate duration = exit time minus entry time.
5. Read the current rate bands from the data store.
6. Compare the duration against the rate bands to determine the fee.
7. Display the duration and fee to the driver.

Fee bands used for this calculation:

Duration	Fee
Up to 30 minutes	Free
Up to 2 hours	Kshs. 50
Up to 4 hours	Kshs. 100
Up to 6 hours	Kshs. 300
Over 6 hours	Kshs. 500

3.4 Payment Module

Collects payment from the driver before the vehicle is allowed to exit.

1. Display the calculated fee to the driver.
2. Driver selects a payment method: M-Pesa, Card, or Cash.
3. If no valid payment method is selected, display an error and stop.

4. Mark the vehicle record as paid and record the chosen payment method.
5. Save the updated record.

3.5 Barrier Control Module

Controls the physical exit barrier based on payment status.

1. Check whether the vehicle record is marked as paid.
2. If paid, open the exit barrier and free the assigned slot.
3. If not paid, keep the barrier closed and display a “Payment required” message.

3.6 Rate Management Module

Allows management to update parking fees without changing the underlying program.

1. Load the current rates from the data store.
2. Display them in an editable form.
3. If any value entered is not a whole number, display an error and stop.
4. If any value entered is negative, display an error and stop.
5. Save the updated rates back to the data store.
6. All future fee calculations immediately use the new rates.

3.7 Audit and Reporting Module

Keeps a permanent record of every payment collected, for reconciliation and VAT.

1. When a payment is confirmed, write a transaction record containing the plate, slot, entry time, exit time, duration, fee, and payment method.
2. Load the full transaction log when the reports page is requested.
3. Sum all fees collected to calculate total revenue.
4. Group totals by payment method.
5. Display an itemised, timestamped list of all transactions.

4. Data Structures and Reasons for Their Use

Data Structure	Used For	Why It's Suitable
Dictionary (Hash Map)	Active vehicles, keyed by plate number	O(1) fast lookup when a car exits
List	All parking slots	Simple to iterate, count, and display in order
List of dictionaries	Transaction log	Preserves order of events; easy to sum and filter for reports

String	Plate numbers, timestamps, payment method	Natural fit for identifiers and readable values
datetime object	Entry and exit time tracking	Built-in time arithmetic for accurate duration calculation
Nested JSON object	Whole system state (rates, slots, vehicles, transactions)	Mirrors structure directly; updates dynamically

5. Dynamic Database Design

The system uses a structured JSON document as its dynamic data store. It is dynamic because the rates, slots, vehicles, and transaction log all update in real time as the system runs.

```
{
  "rates": {
    "free_minutes": 30, "band1_minutes": 120, "band1_fee": 50,
    "band2_minutes": 240, "band2_fee": 100, "band3_minutes": 360,
    "band3_fee": 300, "over_limit_fee": 500
  },
  "slots": [
    {"slot_id": 1, "status": "Free"},
    {"slot_id": 2, "status": "Occupied"}
  ],
  "vehicles": {
    "KDA123A": {
      "slot_id": 2, "entry_time": "2026-09-14T09:00:00",
      "exit_time": null, "fee": null, "paid": false
    }
  },
  "transactions": [
    {
      "plate": "KDB456B", "slot_id": 4,
      "entry_time": "2026-09-14T08:00:00",
      "exit_time": "2026-09-14T09:10:00",
      "duration_minutes": 70, "fee": 50,
      "payment_method": "M-Pesa",
      "paid_at": "2026-09-14T09:11:00"
    }
  ]
}
```

Field descriptions:

- rates — the current fee bands, editable by management and read fresh on every fee calculation

- slot_id — unique identifier for a parking bay
- status — current state of a bay: Free or Occupied
- entry_time / exit_time — timestamps used for duration calculation
- fee — amount computed for the current parking session
- paid — flag checked by the Barrier Control Module before opening the exit barrier
- transactions — a permanent log of every completed, paid session, used for reconciliation and VAT

6. Scope

In scope: entry lane control, slot monitoring and display, slot allocation, duration and fee computation, payment collection, exit barrier control, exception handling, and administrative reporting.

Out of scope for this phase: online pre-booking of bays, valet operations, integration with third-party loyalty schemes, and automated number plate blacklisting.

7. Conclusion

The modules, algorithms, and data structures above address every stated requirement: visual slot display, automated vehicle recording, automatic fee calculation on management-controlled rates, flexible payment collection, barrier control on confirmed payment, and a full audit trail for reconciliation and VAT. This design forms the basis for the functional system, SmartPark KE, implemented in Python with a Flask web interface.